

Review of CS486/686

Wenhu Chen, Pascal Poupart

Lecture 24

Outline

Search Algorithm

Uncertainty Estimation

Machine Learning

Deep Learning

Miscellaneous

Search Algorithm

Uncertainty Estimation

Machine Learning

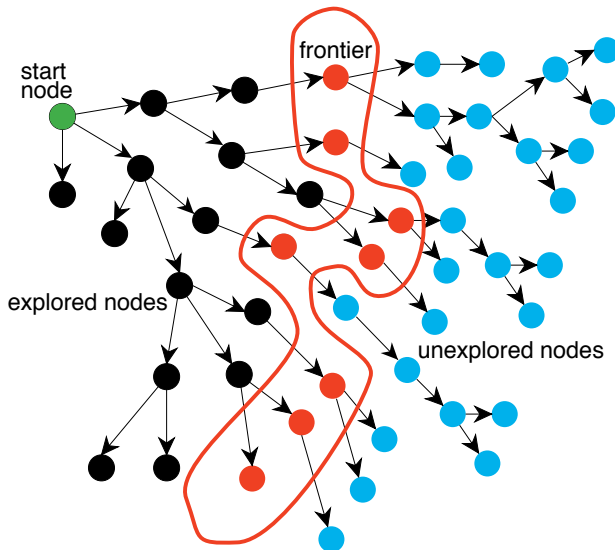
Deep Learning

Miscellaneous

Uninformed Search

- ▶ How to formulate a search problem?
- ▶ What is a search tree?
- ▶ What is generic Search algorithm?
- ▶ What is DFS and what is BFS?
- ▶ What is the space/time complexity of DFS and BFS?
- ▶ What is the iterative deepening space complexity?

The Search Tree



Generic Search Algorithm

Algorithm 1 A Generic Search Algorithm

```
1: procedure SEARCH(Graph, Start node  $s$ , Goal test  $goal(n)$ )
2:   frontier :=  $\{\langle s \rangle\}$ 
3:   while frontier is not empty do
4:     select and remove path  $\langle n_0, \dots, n_k \rangle$  from frontier
5:     if  $goal(n_k)$  then
6:       return  $\langle n_0, \dots, n_k \rangle$ 
7:     for every neighbour  $n$  of  $n_k$  do
8:       add  $\langle n_0, \dots, n_k, n \rangle$  to frontier
9:   return no solution
```

How to use heuristic search?

- ▶ What is LCFS (lowest-cost first)?
- ▶ What is GBFS (lowest-heuristic first)?
- ▶ What is A* search (combination of two)?

A* Search Algorithm

- ▶ Space and Time Complexities.
- ▶ Completeness and Optimality.
- ▶ Admissible Heuristics $\rightarrow$ Optimality
- ▶ Consistent Heuristics $\rightarrow$ Multi-Path Pruning
- ▶ Prove the admissibility and consistency criterion

A^* is Optimal with admissibility constraint

- ▶ Assuming you have many paths in the frontier:
 $(S \rightarrow G : C^*, \dots, S \rightarrow N : C^n)$, and $C^* \leq C^n$.
- ▶ If there a path through N to G has a lower cost of $C' < C^*$.
- ▶ According to admissibility, $C^n \leq C' < C^*$.
- ▶ It's contradictory to our assumption.

Summary of Search Strategies

Strategy	Frontier Selection	Halts?	Space	Time
Depth-first	Last node added	No	Linear	Exp
Breadth-first	First node added	Yes	Exp	Exp
Lowest-cost-first	$\min cost(n)$	Yes	Exp	Exp
Greedy Best-first	$\min h(n)$	No	Exp	Exp
A*	$\min cost(n) + h(n)$	Yes	Exp	Exp

Constraint Satisfaction Problem

- ▶ Why do we need to model the internal structure of the state?
- ▶ What is Backtracking Algorithm?
- ▶ What is Arc consistency Algorithm?
- ▶ AC-3 Algorithm, using Arc consistency to eliminate Arc
- ▶ AC-3 Algorithm complexity
- ▶ Combine Backtracking with AC-3 algorithm

Backtracking Search

Algorithm 2 BACKTRACK(*assignment*, *csp*)

- 1: **if** *assignment* is complete **then return** *assignment*
 - 2: Let *var* be an unassigned variable
 - 3: **for every** *value* in the domain of *var* **do**
 - 4: **if** adding {*var* = *value*} satisfies every constraint **then**
 - 5: add {*var* = *value*} to *assignment*
 - 6: *result* $\leftarrow$ BACKTRACK(*assignment*, *csp*)
 - 7: **if** *result* $\neq$ failure **then return** *result*
 - 8: remove {*var* = *value*} from *assignment* if it was added
 - 9: **return** failure
-

The AC-3 Arc Consistency Algorithm

Algorithm 3 The AC-3 Algorithm

- 1: put every arc in the set S .
 - 2: **while** S is not empty **do**
 - 3: select and remove $\langle X, c(X, Y) \rangle$ from S
 - 4: remove every value in D_X that doesn't have a value in D_Y
 that satisfies the constraint $c(X, Y)$
 - 5: **if** D_X was reduced **then**
 - 6: **if** D_X is empty **then return** false
 - 7: for every $Z \neq Y$, add $\langle Z, c'(Z, X) \rangle$ to S
 - return** true
-

Local Search

- ▶ Why do we need local search?
- ▶ How do we perform greedy descent?
- ▶ How can we avoid local minima?
- ▶ What is Simulated Annealing?
- ▶ What is Genetic Algorithm?

Simulated Annealing Algorithm

Algorithm 4 Simulated Annealing

```
1: current  $\leftarrow$  initial-state
2:  $T \leftarrow$  a large positive value
3: while  $T > 0$  do
4:   next  $\leftarrow$  a random neighbour of current
5:    $\Delta C \leftarrow \text{cost}(\text{next}) - \text{cost}(\text{current})$ 
6:   if  $\Delta C < 0$  then
7:     current  $\leftarrow$  next
8:   else
9:     current  $\leftarrow$  next with probability  $p = e^{\frac{-\Delta C}{T}}$ 
10:  decrease  $T$ 
11: return current
```

Search Algorithm

Uncertainty Estimation

Machine Learning

Deep Learning

Miscellaneous

Independence

- ▶ What is unconditional independence?
- ▶ What is conditional independence?
- ▶ What is chain rule/product rule/sum rule/bayes rule?
- ▶ Universal approach to calculate a probability.

Independence

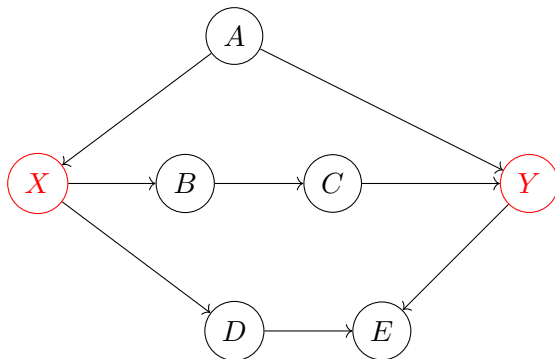
- ▶ Given joint probability distribution, derive the independence step by step.
- ▶ How are independence verified quantitatively.
- ▶ Why do we need to use Bayesian Networks?
- ▶ How can we compute joint probability over a Bayesian Network?

D-Separation

- ▶ What is D-Separation Rule 1?
- ▶ What is D-Separation Rule 2?
- ▶ What is D-Separation Rule 3?
- ▶ How do you apply these D-Separation rules to understand independence between different nodes?

D-Separation

- ▶ Un-directed paths between X and Y .
- ▶ Multiple paths need to be considered if they exist.
- ▶ One of the nodes on all the paths blocking the connection.



Constructing Bayesian Network

- ▶ Identify all the (conditional) independence relationships
- ▶ Pick an order
- ▶ Add nodes to the graph
- ▶ Pick the minimum subset as parents according to the (conditional) independence relationships
- ▶ Form a Bayesian Network

Search Algorithm

Uncertainty Estimation

Machine Learning

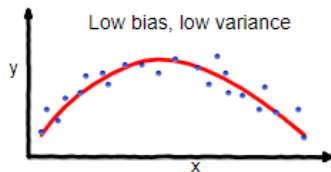
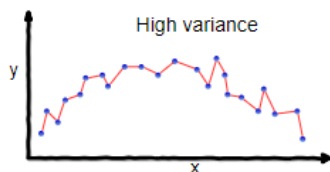
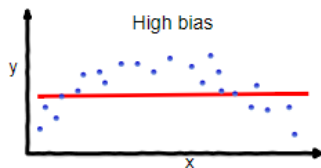
Deep Learning

Miscellaneous

Supervised Learning

- ▶ Classification vs. Regression
- ▶ Cross-Validation
- ▶ How to avoid Over-fitting?
- ▶ How to derive Bias-Variance equation?
- ▶ Trade-offs between bias and variance.

Bias-Variance Trade-off



Bias-Variance Trade-off

So the expected squared error at a point x is

$$Err(x) = E \left[(Y - \hat{f}(x))^2 \right]$$

The $Err(x)$ can be further decomposed as

$$Err(x) = \left(E[\hat{f}(x)] - f(x) \right)^2 + E \left[\left(\hat{f}(x) - E[\hat{f}(x)] \right)^2 \right] + \sigma_e^2$$

$$Err(x) = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Neural Networks I

- ▶ Approximating Different Gate function with Neural Network.
- ▶ What is activation function and what qualifies as activation functions?

Neural Networks II

- ▶ What is gradient descent?
- ▶ What is loss function?
- ▶ What's the rule of Backward propagation?
- ▶ How to perform backpropagation on 1 or 2 layers of neural network?
- ▶ Understand the computation/memory complexity of backpropagation

The recursive relationship

Backward Propagation Algorithm:

- ▶ Initialize W_i for all the layers (from 1 to n).
- ▶ Feedforward x into neural network and save intermediate values $g(x^{(1)}), g(x^{(2)}), \dots$.
- ▶ Compute $\delta_n = \frac{\partial E}{\partial z} \cdot \frac{\partial g(x^{(n)})}{\partial x^{(n)}}$.
- ▶ For $i = n \rightarrow 2$; do
 - ▶ $\delta_{i-1} = \delta_i \cdot W_i^T \cdot \frac{\partial g(x^{(i-1)})}{\partial x^{(i-1)}}$
 - ▶ Compute $\frac{\partial E}{\partial W_i} = g(x^{(i-1)}) \otimes \delta_i$
- ▶ $\frac{\partial E}{\partial W_1} = x^0 \otimes \delta_1$, where x^0 is the input.
- ▶ Obtain all $\frac{\partial E}{\partial W_i}$ for gradient descent.

Neural Networks III

- ▶ What is stochastic gradient descent?
- ▶ What is momentum method?
- ▶ What is adaptive method?
- ▶ Understand how to use Adam optimizer.

Adam: ADaptive Moments

Algorithm 5 ADaptive Moments

Require: Learning Rate ϵ , Decay rates $\rho_1, \rho_2, \theta, \delta$

- 1: Initialize $\mathbf{s} = 0$, $\mathbf{r} = 0$, time step $t = 0$
 - 2: **while** stopping criteria not met **do**
 - 3: Sample example $(x^{(i)}, y^{(i)})$ from training set
 - 4: Compute gradient estimate: $\hat{g} \leftarrow +\nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$
 - 5: $t \leftarrow t + 1$
 - 6: Update: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \hat{g}$
 - 7: Update: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \hat{g} \odot \hat{g}$
 - 8: Correct Biases: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}, \hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$
 - 9: Compute Update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$
 - 10: Apply Update: $\theta \leftarrow \theta + \Delta \theta$
-

Search Algorithm

Uncertainty Estimation

Machine Learning

Deep Learning

Miscellaneous

Unsupervised Learning

- ▶ How to do K-means clustering?
- ▶ What is Principled component Analysis?
- ▶ Understanding the two algorithms of PCA.
- ▶ Draw the connection between the two algorithms of PCA.
- ▶ What is auto-encoder?
- ▶ What is the optimization goal of GANs?

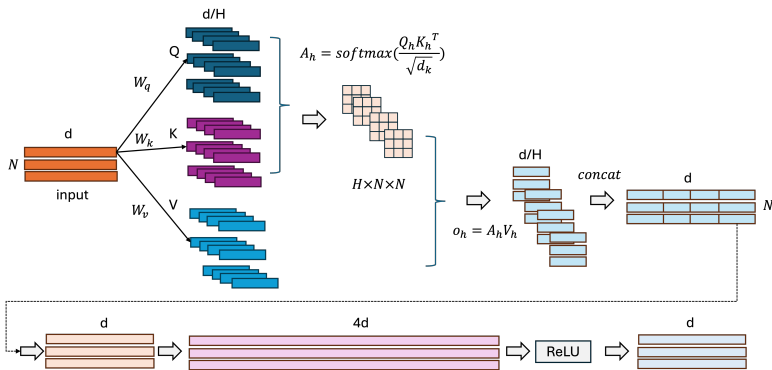
CNN & RNNs

- ▶ Understand RNN architecture.
- ▶ Perform backward propagation through time with two steps.
- ▶ Understand CNN architecture.
- ▶ Perform backward propagation on 2x2 kernel on 3x3 image.

Transformers

- ▶ Understand the motivation of self-attention.
- ▶ Understand the difference between self-attention and cross-attention.
- ▶ Understand the computation flow of a single-layer self-attention.
- ▶ Understand the computation and memory complexity of single-layer self-attention.

Self-Attention Layer



Search Algorithm

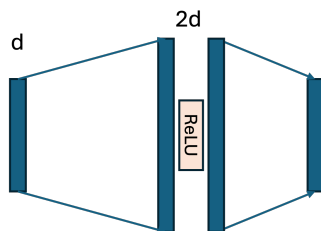
Uncertainty Estimation

Machine Learning

Deep Learning

Miscellaneous

Neural Networks Computation Complexity (Forward)



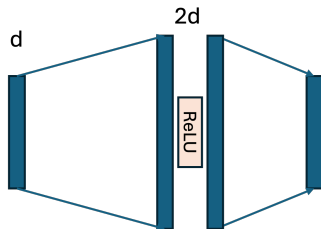
- ▶ Computation cost is mostly coming from matrix multiplication. Activations have negligible computation.
- ▶ In the forward propagation, there will be $B \times d \times 2d$ and $B \times 2d \times d$ matrix multiplications.
- ▶ Total cost is $4Bd^2$.

Neural Networks Computation Complexity (Backward)

Assuming we have one layer of d by d' dimensions (weight matrix of W), where the input is h_t and the output is h_{t+1} .

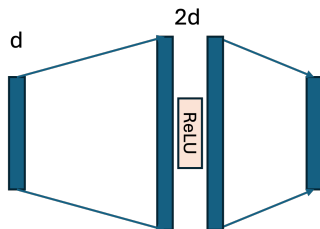
- ▶ We need $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial h_{t+1}} \cdot h_t$. This operation consists of a matrix multiplication of $\mathbb{R}^{B \times d' \times 1}$ and $\mathbb{R}^{B \times 1 \times d}$, which consumes Bdd' multiplication.
- ▶ We also need to use chain rule to compute $\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_{t+1}} \cdot W$. This operation consists of a matrix multiplication of $\mathbb{R}^{B \times d'}$ and $\mathbb{R}^{d' \times d}$, which consumes another Bdd' multiplication.
- ▶ The forward pass is Bdd' , then the backward is roughly doubling it to $2Bdd'$. Therefore, in order to know the backward computation, then we can just double that.
- ▶ In the previous network, the total will be $8Bd^2$.

Neural Networks Memory Complexity (Forward)



- ▶ Memory complexity is the peak memory we use.
- ▶ In the forward scenario, let's say that we don't do backprop and we don't want to cache anything.
- ▶ We can implement everything in-place. The memory cost is basically the largest intermediate. It's $2Bd$ in this case.
- ▶ Model weights have to stay in the memory, therefore, the total is $4d^2 + 2Bd$.

Neural Networks Memory Complexity (Backward)



- ▶ We need to cache every hidden vectors and input and output in the neural network. This is for preparing backward propagation. Therefore, the total is $Bd + B2d + B2d + Bd = 6Bd$.
- ▶ We need to store all the weights in the memory, which is $4d^2$.
- ▶ If we are using Adam, there will first-order and second-order momentum, which are exactly the same size of weight $8d^2$.
- ▶ The total is $12d^2 + 6Bd$.

Exam

- ▶ Non-programmable calculator.
- ▶ Single-sided A4 Study Note.
- ▶ A total of 8 problems.
- ▶ 5 problems from Wenhui.
- ▶ 3 problems from Pascal.
- ▶ 100 marks in total.
- ▶ If you can't attend the final exam, we will mark as INC to allow you to take the final in the following term.
- ▶ Pass the exam to pass the course.

Lecture 24: Other AI Courses

CS486/686 Intro to Artificial Intelligence

Pascal Poupart
David R. Cheriton School of Computer Science
CIFAR AI Chair at Vector Institute



Other AI Courses

- CS480/680: Intro to Machine Learning
 - Support vector machines, logistic regression, Gaussian processes, linear regression, CNNs, RNNs, transformers, variational autoencoders, generative adversarial networks, graph neural networks, normalizing flows, diffusion models, bagging/boosting, transfer learning, fairness, etc.
- CS485/685: Learning theory
- CS484/684: Computer vision
- CS479: Biologically plausible neural networks
- CS794: Optimization for Data Science
- CS885: Reinforcement Learning (Winter 2025, instructor: Pascal Poupart)
- CS886: Advanced topics in AI
 - Graph neural networks, NLP, Vision, multiagent systems, robust ML, learning theory